

Laboratorio 4 - Data Science

Pivi Riccardo

Dicembre 2025

1 Esercizio 1: La Somma - Problema di Regressione

1.1 I dati:

Si sono create 10000 coppie di numeri interi positivi appartenenti all'intervallo $[0, 999]$ e si sono calcolate le somme delle coppie:

```
1 X=np.random.randint(size=(10000,2),high=999,low=0)
2 SumX = np.sum(X,axis=1)
```

Si definisce la classe Dataset di PyTorch:

```
1 class Dataset(torch.utils.data.Dataset):
2     def __init__(self, X, Y):
3         self.x = torch.tensor(X,dtype=torch.float32)
4         self.y = torch.tensor(Y,dtype=torch.float32).unsqueeze(1)
5         self.len = self.x.shape[0]
6
7     def __len__(self):
8         return self.len
9
10    def __getitem__(self, idx):
11        return self.x[idx], self.y[idx]
```

1.2 La rete:

Si definisce un modello sequenziale con due layers nascosti densi, aventi 64 neuroni ciascuno e con attivazione ReLU, e un layer di output denso con un solo neurone:

```
1 class DenseNN(nn.Module):
2     def __init__(self):
3         super(DenseNN, self).__init__()
4         self.fc1 = nn.Linear(2, 64)
5         self.fc2 = nn.Linear(64, 64)
6         self.fc3 = nn.Linear(64, 1)
7
8     def forward(self, x):
9         x = torch.relu(self.fc1(x)) # Appliciamo ReLU dopo il primo strato
10        x = torch.relu(self.fc2(x)) # Appliciamo ReLU dopo il secondo strato
11        x = self.fc3(x)
12        return x
```

1.3 Configurazione del processo di addestramento:

Il processo di addestramento è stato configurato come richiesto con ottimizzatore Adam e Loss MSE. Inoltre è stato scelto un learning rate di 0.0001, poichè ha portato a risultati soddisfacenti nell'apprendimento delle somme:

```

1 model = DenseNN().to(device)
2 criterion = nn.MSELoss() # Funzione loss adatta a problemi di regressione
3 optimizer = optim.Adam(model.parameters(), lr=0.0001) # Ottimizzatore Adam

```

Si vuole addestrare il modello sull'80% dei dati, mentre lo si è validato sul 20% :

```

1 perc=0.8 # percentuale di dati per il training
2 n = int(10000 * perc)
3 train_dataset = Dataset(X[:n,:], SumX[:n])
4 test_dataset = Dataset(X[n:,:], SumX[n:])

```

Si è utilizzato un `batch_size = 64`:

```

1 # Creiamo i DataLoader per gestire i batch
2 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
3 test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

```

1.4 Addestramento:

Si è addestrato il modello iterando per 100 epoche:

```

1 epochs = 100
2 train_losses = []
3
4 for epoch in range(epochs):
5     model.train() # Mettiamo il modello in modalit training
6     running_loss = 0.0
7
8     for inputs, labels in train_loader:
9         inputs, labels = inputs.to(device), labels.to(device) # Spostiamo i dati sul dispositivo
10
11         optimizer.zero_grad() # Azzeriamo i gradienti
12         outputs = model(inputs) # Forward pass
13         loss = criterion(outputs, labels) # Calcoliamo la perdita
14         loss.backward() # Backward pass
15         optimizer.step() # Aggiorniamo i pesi
16
17         running_loss += loss.item() * inputs.size(0)
18
19     epoch_loss = running_loss / len(train_loader.dataset)
20     train_losses.append(epoch_loss)
21     print(f"Epoch_{epoch+1}/{epochs}, Loss: {epoch_loss:.4f}")

```

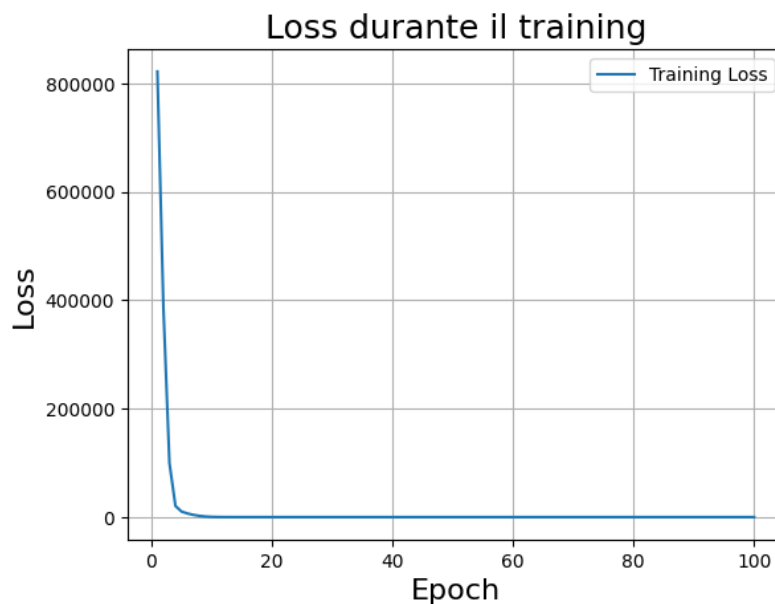


Figura 1: Grafico che mostra l'andamento della Loss durante il training. Sull'asse delle ordinate la Loss, mentre sulle ascisse l'epoca.

1.5 Validazione e Confronti:

```
1 from sklearn.metrics import r2_score
2
3 model.eval()
4 all_outputs = []
5 all_labels = []
6
7 with torch.no_grad():
8     for inputs, labels in test_loader:
9         inputs, labels = inputs.to(device), labels.to(device)
10        outputs = model(inputs)
11
12        all_outputs.append(outputs.cpu()) # portiamo su CPU
13        all_labels.append(labels.cpu())
14
15 # concatenare tutti i batch
16 all_outputs = torch.cat(all_outputs).numpy()
17 all_labels = torch.cat(all_labels).numpy()
18
19 # calcolare R^2
20 r2 = r2_score(all_labels, all_outputs)
21 print("R^2 sul set di test:", r2)
```

La validazione ha dato come risultato un $R^2_{sulsetdite} = 0.99999970$ che è estremamente vicino ad uno.

Si procede ad utilizzare il modello addestrato con degli esempi:

```
1 Z=np.random.randint(size=(1,2),high=999,low=0)
2 print(Z)
3 realSumZ = Z[0,0]+Z[0,1]
4 modelSumZ = model(torch.tensor(Z,dtype=torch.float32).to(device))
5 print("Real_sum:", realSumZ)
6 print("Model_sum:", modelSumZ.item())
```

Si fornisce un esempio: $Z = [121, 296]$, Real sum: 417 , Model sum: 416.99591064453125 .

Il modello ha predetto un valore della somma molto vicino a quello reale come ci aspettiamo essendo i numeri nel range dei dati forniti per l'allenamento.

Si ripete il codice precedente con numeri interi positivi, ma di valore numerico di ordini di grandezza più alti:

```
1 Z=np.random.randint(size=(1,2),high=999999,low=0)
```

Si fornisce un esempio: $Z = [123402, 199625]$, Real sum: 323027 , Model sum: 322878.46875 .

La predizione del modello è corretta fino alla seconda cifra significativa.

Si ripete il codice precedente con numeri interi sia positivi che negativi:

```
1 Z=np.random.randint(size=(1,2),high=999,low=-999)
```

Si fornisce un esempio: $Z = [948, -801]$, Real sum: 147, Model sum: 505.40228271484375 .

Il modello in questo caso sbaglia il risultato fin dalla prima cifra significativa.

```
1 H = [[5, 7], [100, 200], [400, 200], [999, 999]]
2 realSumH = np.sum(H, axis=1)
3 modelSumH = model(torch.tensor(H,dtype=torch.float32).to(device))
4 print("Real_sum:", realSumH)
5 print("Model_sum:", modelSumH)
```

Fornendo le coppie H di numeri interi nel range utilizzato per l'allenamento del modello possiamo confrontare anche in questo caso le predizioni con la somma reale.

Real sum: [12, 300, 600, 1998] , Model sum: [12.0879, 300.0309, 600.0102, 1997.7808] .

Notiamo - come detto in precedenza - che in questo range il modello fa delle predizioni molto vicine al risultato reale.

2 Esercizio 2: Il Segno - Problema di Classificazione

2.1 I dati:

Si sono create 10000 coppie di numeri interi appartenenti all'intervallo $[-999, 999]$ e si sono calcolate le somme delle coppie:

```
1 X=np.random.randint(size=(10000,2),high=999,low=-999)
2 SumX = np.sum(X,axis=1)
```

I risultati delle somme sono stati classificati fra le 3 classi (negativo, nullo, positivo):

```
1 y = [] # tre classi di etichette: negativo, zero, positivo
2 for i in range(len(SumX)):
3     if SumX[i] < 0:
4         y.append([1,0,0])
5     if SumX[i] == 0:
6         y.append([0,1,0])
7     if SumX[i] > 0:
8         y.append([0,0,1])
```

Si definisce la classe Dataset di PyTorch:

```
1 class Dataset(torch.utils.data.Dataset):
2     def __init__(self, X, Y):
3         self.x = torch.tensor(X,dtype=torch.float32)
4         self.y = torch.tensor(Y,dtype=torch.float32)
5         self.len = self.x.shape[0]
6
7     def __len__(self):
8         return self.len
9
10    def __getitem__(self, idx):
11        return self.x[idx], self.y[idx]
```

2.2 La rete:

Si definisce un modello sequenziale con due layers nascosti densi, aventi 64 neuroni ciascuno e con attivazione ReLU, e un layer di output denso con tre neuroni (uno per ogni classe):

```
1 class DenseNN(nn.Module):
2     def __init__(self):
3         super(DenseNN, self).__init__()
4         self.fc1 = nn.Linear(2, 64)
5         self.fc2 = nn.Linear(64, 64)
6         self.fc3 = nn.Linear(64, 3)
7
8     def forward(self, x):
9         x = torch.relu(self.fc1(x)) # Appliciamo ReLU dopo il primo strato
10        x = torch.relu(self.fc2(x)) # Appliciamo ReLU dopo il secondo strato
11        x = self.fc3(x)
12        return x
```

2.3 Configurazione del processo di addestramento:

Il processo di addestramento è stato configurato con ottimizzatore Adam e come Loss la CrossEntropy. Inoltre è stato scelto un learning rate di 0.0001, poichè ha portato a risultati soddisfacenti nell'apprendimento del segno della somma:

```
1 model = DenseNN().to(device) # Spostiamo il modello sul dispositivo scelto
2 criterion = nn.CrossEntropyLoss() # Funzione di perdita per problemi di classificazione
3 optimizer = optim.Adam(model.parameters(), lr=0.0001) # Ottimizzatore Adam
```

Si vuole addestrare il modello sull'80% dei dati, mentre lo si è validato sul 20% :

```
1 perc =0.8 # percentuale di dati per il training
2 n = int(10000 * perc)
3 train_dataset = Dataset(X[:n,:], y[:n])
4 test_dataset = Dataset(X[n:,:], y[n:])
```

Si è utilizzato un `batch_size = 64`:

```
1 # Creiamo i DataLoader per gestire i batch
2 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
3 test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

2.4 Addestramento:

Si è addestrato il modello iterando per 100 epoche:

```
1 epochs = 100
2 train_losses = []
3
4 for epoch in range(epochs):
5     model.train() # Mettiamo il modello in modalit training
6     running_loss = 0.0
7
8     for inputs, labels in train_loader:
9         inputs, labels = inputs.to(device), labels.to(device) # Spostiamo i dati sul dispositivo
10
11         optimizer.zero_grad() # Azzeriamo i gradienti
12         outputs = model(inputs) # Forward pass
13         loss = criterion(outputs, labels) # Calcoliamo la perdita
14         loss.backward() # Backward pass
15         optimizer.step() # Aggiorniamo i pesi
16
17         running_loss += loss.item() * inputs.size(0)
18
19     epoch_loss = running_loss / len(train_loader.dataset)
20     train_losses.append(epoch_loss)
21     print(f"Epoch_{epoch+1}/{epochs}, Loss: {epoch_loss:.4f}")
```

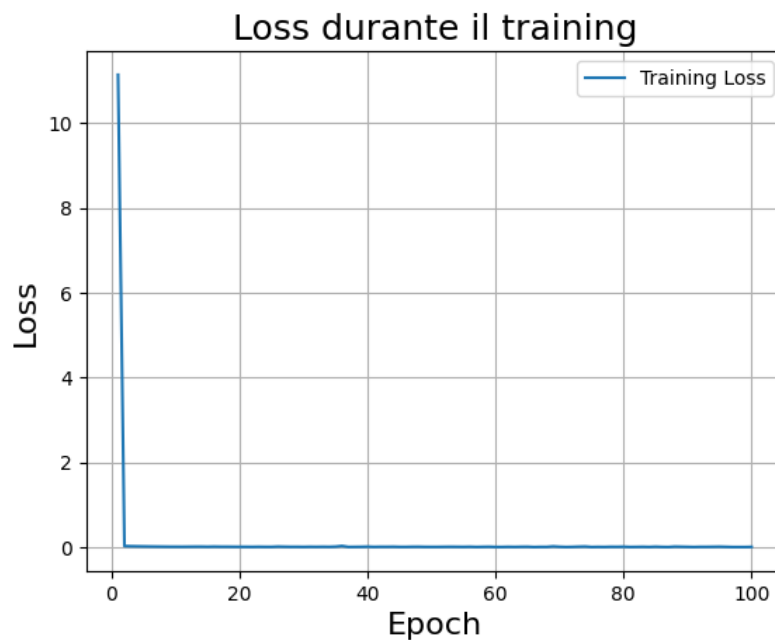


Figura 2: Grafico che mostra l'andamento della Loss durante il training. Sull'asse delle ordinate la Loss, mentre sulle ascisse l'epoca.

2.5 Accuratezza:

```
1 model.eval() # Mettiamo il modello in modalit valutazione
2 correct = 0
3 total = 0
4 misclassified = []
5
6 with torch.no_grad():
7     for inputs, labels in test_loader:
```

```

8     inputs, labels = inputs.to(device), labels.to(device)
9     outputs = model(inputs)
10    _, predicted = torch.max(outputs, 1) # Prevediamo la classe con probabilit pi alta
11    #print(labels)
12    true_class = torch.argmax(labels,axis=1)
13    #print(predicted,true_class)
14    total += labels.size(0)
15    correct += (predicted == true_class).sum().item()
16
17 accuracy = 100 * correct / total
18 print(f'Accuracy sul test set: {accuracy:.2f}%')

```

La validazione ha dato come risultato una $Accuracy_{sul\ test\ set}$: 99.90% molto prossimo al 100% .

2.6 Classificazione o Regressione?

Se al posto della somma si utilizza come etichetta $\text{np.sign}(\text{inputdata[:,0]} + \text{inputdata[:,1]})$, il problema cambia natura. L'output non è più continuo, ma assume solo valori discreti, quindi la funzione da apprendere è discontinua.

Il codice dell'esercizio 1 è stato progettato per risolvere un problema di regressione, in cui la rete neurale deve apprendere una funzione continua: la somma di due numeri reali. In questo contesto l'uso di un neurone di output lineare e della funzione di perdita MSE (mean squared error) è appropriato, perché l'obiettivo è approssimare il più accuratamente possibile un valore reale continuo. La MSE non funzionerebbe con la funzione segno perché cerca di minimizzare la distanza tra valori continui, ma il segno è una funzione discontinua con solo 3 valori discreti (-1, 0, 1). Inoltre, la discontinuità della funzione segno implica che piccole variazioni degli input vicino alla frontiera $x_1 + x_2 = 0$ producono grandi variazioni nell'output, rendendo l'ottimizzazione con MSE inefficiente. Per questo è necessaria una loss per problemi di classificazione come CrossEntropy.